

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Analytical Problem Solving: 40%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

IM. Varshith Reddy(192524174) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M. Varshith Reddy

Annexure A Analytical Problem Solving

1. RSA Encryption Analysis

Given $p=11$, $q=13$

- a. Compute n $\phi(n)$
- b. Choose $e=7$ and find the private key d
- c. Encrypt the message $M=9$
- d. Decrypt the ciphertext and verify the result
- e. Analyze how changing e affects security

2. Block Cipher Mode

Evaluation Given plaintext blocks:

$P=[1010,1010,1111,1010]$, Key = $K=1100$

- a. Encrypt using ECB and CBC modes (Assume IV = 0000)
- b. Compare the ciphertext outputs
- c. Analyze which mode hides patterns better and explain why

3. DES vs 3-DES vs Blowfish

Performance Study Given:

- a. DES key size = 56 bits, time = 2 ms/block
- b. 3-DES key size = 168 bits, time = 6 ms/block
- c. Blowfish key size = 128 bits, time = 1 ms/block

For 1000 blocks of data:

- i. Calculate total encryption time for each algorithm
- ii. Analyze trade-offs between security and performance
- iii. Recommend the best algorithm for secure real-time communication

4. Diffie-Hellman Key

Exchange Analysis Given:

- a. $p=23$, $g=5$
- b. User A private key = 6
- c. User B private key = 15
- d. Compute public keys for A and B
- e. Compute the shared secret key
- f. Demonstrate how a Man-in-the-Middle attacker could alter the exchange
- g. Suggest a secure method to prevent the attack

5. ECC vs RSA Comparison

Given:

- a. RSA key size = 1024 bits
- b. ECC key size = 160 bits (equivalent security)
- c. Device processing capacity = limited (IoT device)
- d. Analyze computational cost for both systems
- e. Estimate which consumes less power and memory
- f. Justify which cryptosystem is better for the given scenario with reasoning

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Method Selection	20%	Selects most appropriate method/formula with	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
		strong justification			
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. RSA Encryption Analysis ($p = 11$, $q = 13$)

a. Compute n and $\varphi(n)$

RSA starts by choosing two prime numbers, p and q .

- $n = p \times q = 11 \times 13 = \mathbf{143}$
- $\varphi(n)$ (Euler's totient function) $= (p - 1)(q - 1) = 10 \times 12 = \mathbf{120}$

$n = 143$ becomes the modulus used in both the public and private keys. $\varphi(n) = 120$ tells us how many integers between 1 and 143 are coprime to 143 — this value is what e and d must be mathematically related to.

b. Choose $e = 7$, find private key d

First verify e is valid: e must satisfy $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$.

$\gcd(7, 120)$: $120 = 17 \times 7 + 1$, $7 = 7 \times 1 + 0 \rightarrow \gcd = 1$ ✓ valid.

Now find d such that: $\mathbf{e \times d \equiv 1 \pmod{\varphi(n)}}$, i.e., $7d \equiv 1 \pmod{120}$

Using the Extended Euclidean Algorithm:

$$120 = 17 \times 7 + 1$$

$$7 = 7 \times 1 + 0$$

Back-substitute:

$$1 = 120 - 17 \times 7$$

This means $-17 \times 7 \equiv 1 \pmod{120}$, so $d \equiv -17 \pmod{120} \rightarrow d = -17 + 120 = \mathbf{103}$

Verify: $7 \times 103 = 721 = 6 \times 120 + 1$ ✓ ($721/120 = 6.008\dots$, $6 \times 120 = 720$, remainder 1)

- Public key: $(e, n) = \mathbf{(7, 143)}$
- Private key: $(d, n) = \mathbf{(103, 143)}$

c. Encrypt $M = 9$

$$C = M^e \pmod{n} = 9^7 \pmod{143}$$

Compute step by step using repeated squaring/multiplication:

- $9^1 = 9$
- $9^2 = 81$
- $9^3 = 9^2 \times 9 = 81 \times 9 = 729 \text{ mod } 143 \rightarrow 729 - 5 \times 143 = 729 - 715 = \mathbf{14}$
- $9^4 = 9^3 \times 9 = 14 \times 9 = 126$
- $9^5 = 9^4 \times 9 = 126 \times 9 = 1134 \text{ mod } 143 \rightarrow 1134 - 7 \times 143 = 1134 - 1001 = \mathbf{133}$
- $9^6 = 9^5 \times 9 = 133 \times 9 = 1197 \text{ mod } 143 \rightarrow 1197 - 8 \times 143 = 1197 - 1144 = \mathbf{53}$
- $9^7 = 9^6 \times 9 = 53 \times 9 = 477 \text{ mod } 143 \rightarrow 477 - 3 \times 143 = 477 - 429 = \mathbf{48}$

Ciphertext C = 48

d. Decrypt and verify

$$M = C^d \text{ mod } n = 48^{103} \text{ mod } 143$$

Direct computation of 48^{103} is tedious by hand, so we use the **Chinese Remainder Theorem (CRT)** with $p = 11$ and $q = 13$.

Step 1 — reduce exponent mod $(p-1)$ and $(q-1)$:

- $d_p = d \text{ mod } (p-1) = 103 \text{ mod } 10 = 3$
- $d_q = d \text{ mod } (q-1) = 103 \text{ mod } 12 = 7$

Step 2 — reduce base mod p and mod q :

- $48 \text{ mod } 11 = 4$
- $48 \text{ mod } 13 = 9$

Step 3 — compute:

- $M_p = 4^3 \text{ mod } 11 = 64 \text{ mod } 11 = 9$
- $M_q = 9^7 \text{ mod } 13$:
 - $9^2 = 81 \text{ mod } 13 = 3$
 - $9^3 = 9^2 \times 9 = 3 \times 9 = 27 \text{ mod } 13 = 1$
 - $9^6 = (9^3)^2 = 1^2 = 1$
 - $9^7 = 9^6 \times 9 = 1 \times 9 = \mathbf{9}$

Step 4 — combine using CRT (both residues equal 9, so $M = 9$ directly):

Since $M_p = 9$ and $M_q = 9$, and $9 < 143$ satisfies both congruences, **$M = 9$**

Verification: Decrypted message (9) = original plaintext (9) ✓ **RSA correctness confirmed**

e. Effect of changing e on security

- **Small e (e.g., e = 3):** Makes encryption computation faster, but is dangerous. If the same message is broadcast to multiple recipients with different n but the same small e, an attacker can use the **Håstad broadcast attack** (via CRT) to recover the plaintext without factoring n. Small e without proper OAEP padding is also vulnerable to **Coppersmith's attack**.
- **Large e (e.g., e = 65537 = 2¹⁶+1):** This is the industry-standard choice because it's large enough to resist low-exponent attacks while still being efficient to compute (only 17 modular multiplications due to its binary form 100000000000000001).
- **e too close to φ(n) or badly chosen d:** If e is chosen such that the resulting d is small, the system becomes vulnerable to **Wiener's attack**, which can efficiently recover a small d using continued fractions.
- **Conclusion:** e = 7 works mathematically here but would be considered weak in practice for real systems; the choice of e affects both computational efficiency (smaller e → faster encryption) and security (very small or poorly paired e/d → vulnerable to known cryptanalytic attacks).

2. Block Cipher Mode Evaluation

Given: P = [1010, 1010, 1111, 1010], K = 1100, IV = 0000

Encryption function: $E(X) = X \oplus K$ (simple XOR cipher, standard for this style of textbook problem)

a(i). ECB (Electronic Codebook) Mode

In ECB, each block is encrypted **independently** using the same key:

$$C_i = E(P_i) = P_i \oplus K$$

Block	Plaintext	XOR with K=1100	Ciphertext
P1	1010	$1010 \oplus 1100$	0110
P2	1010	$1010 \oplus 1100$	0110

P3	1111	$1111 \oplus 1100$	0011
P4	1010	$1010 \oplus 1100$	0110

ECB Ciphertext: [0110, 0110, 0011, 0110]

a(ii). CBC (Cipher Block Chaining) Mode

In CBC, each plaintext block is XORed with the **previous ciphertext block** before encryption. $C_0 = IV = 0000$.

$$C_i = E(P_i \oplus C_{i-1}) = (P_i \oplus C_{i-1}) \oplus K$$

Step	Computation	Result
C1	$(P_1 \oplus C_0) \oplus K =$ $(1010 \oplus 0000) \oplus 110$ $0 = 1010 \oplus 1100$	0110
C2	$(P_2 \oplus C_1) \oplus K =$ $(1010 \oplus 0110) \oplus 1100$ $= 1100 \oplus 1100$	0000
C3	$(P_3 \oplus C_2) \oplus K =$ $(1111 \oplus 0000) \oplus 1100$ $= 1111 \oplus 1100$	0011
C4	$(P_4 \oplus C_3) \oplus K =$ $(1010 \oplus 0011) \oplus 1100$ $= 1001 \oplus 1100$	0101

CBC Ciphertext: [0110, 0000, 0011, 0101]

b. Comparing the outputs

Block	Plaintext	ECB Output	CBC Output
1	1010	0110	0110
2	1010	0110	0000
3	1111	0011	0011
4	1010	0110	0101

- In **ECB**, blocks 1, 2, and 4 – all identical plaintext (1010) – produce **identical ciphertext (0110)**. Anyone observing the ciphertext can immediately tell those three blocks were the same, even without knowing what the data actually is.
- In **CBC**, the same three identical plaintext blocks produce **three different ciphertext values** (0110, 0000, 0101) because each block's encryption depends on the ciphertext of the block before it – a chaining/feedback effect.

c. Which mode hides patterns better, and why

CBC hides patterns better.

Reasoning:

- ECB treats every block completely independently, so identical plaintext always maps to identical ciphertext under a fixed key. This leaks structural information – repeating patterns in the plaintext (like repeated headers, blank sections, or images with large uniform color blocks) remain visible in the ciphertext. This is the classic "ECB penguin" problem, where encrypted images still show recognizable outlines.
- CBC introduces dependency between blocks via the XOR with the previous ciphertext (chained from the IV). This means the same plaintext block encrypted at different positions (or in different messages, given a random IV) will almost always produce different

ciphertext, effectively randomizing the output and destroying any plaintext pattern correlation.

- This is why **ECB is not recommended for encrypting more than one block of data** in real-world systems, while CBC (or better, authenticated modes like GCM) is standard practice.

3. DES vs 3-DES vs Blowfish Performance Study

Given data:

Algorithm	Key size	Time per block
DES	56 bits	2 ms
3-DES	168 bits	6 ms
Blowfish	128 bits	1 ms

i. Total encryption time for 1000 blocks

- **DES:** $1000 \text{ blocks} \times 2 \text{ ms/block} = 2000 \text{ ms} = 2 \text{ seconds}$
- **3-DES:** $1000 \text{ blocks} \times 6 \text{ ms/block} = 6000 \text{ ms} = 6 \text{ seconds}$
- **Blowfish:** $1000 \text{ blocks} \times 1 \text{ ms/block} = 1000 \text{ ms} = 1 \text{ second}$

Summary table:

Algorithm	Key size	Time/block	Total time (1000 blocks)	Relative speed
DES	56-bit	2 ms	2 s	2× slower than Blowfish
3-DES	168-bit	6 ms	6 s	6× slower than

				Blowfish
Blowfish	128-bit	1 ms	1 s	Fastest

ii. Trade-off analysis between security and performance

- **DES (56-bit key, 2s total):**

- *Security:* Weakest of the three. A 56-bit key space (2^{56} possibilities) can be brute-forced with modern hardware/cloud computing in a matter of hours to days – DES was officially broken and deprecated by NIST decades ago.
- *Performance:* Moderate speed, but its weak security makes speed irrelevant for anything sensitive.

- **3-DES (168-bit key, 6s total):**

- *Security:* Much stronger than DES (effective security ~112 bits due to meet-in-the-middle attacks, not the full 168 bits), and considered adequate for legacy systems, but NIST has deprecated it for new applications (post-2023) in favor of AES.
- *Performance:* Slowest of the three – applies the DES algorithm three times sequentially, tripling (or more) the computational overhead. This makes it a poor fit for high-throughput or real-time systems.

- **Blowfish (128-bit key, 1s total):**

- *Security:* Strong for most practical purposes – no known effective cryptanalytic attack against the full 16-round Blowfish. 128-bit key space is computationally infeasible to brute-force with current technology.
- *Performance:* Fastest of the three, using a simple Feistel structure with precomputed S-boxes, making it efficient in both software and hardware.

- **Overall trade-off:** There is a clear inverse relationship between the number of encryption rounds/operations (which strengthens security) and speed. DES sacrifices security for moderate speed; 3-DES sacrifices speed for improved (but not top-tier) security; Blowfish achieves a strong balance – good security **and** the best performance of the three.

iii. Recommendation for secure real-time communication

Blowfish is the recommended choice.

Justification:

- Real-time communication (VoIP, video calls, live chat, streaming) requires **low latency** — every millisecond of encryption/decryption overhead adds to perceptible delay.
- Blowfish provides the **lowest encryption time** (1 s for 1000 blocks vs. 2 s for DES and 6 s for 3-DES) while still offering a **128-bit key**, which is considered cryptographically strong (no practical brute-force or structural attack exists against it).
- DES is disqualified due to its broken 56-bit key security, regardless of speed.
- 3-DES, while secure, is too slow (6× Blowfish) for real-time constraints and is being phased out industry-wide.
- *(Note: In modern real-world systems, AES would typically be preferred over all three due to hardware acceleration (AES-NI) and its status as the current NIST standard — but strictly among the three algorithms given, Blowfish is the best fit.)*

4. Diffie-Hellman Key Exchange Analysis

Given: $p = 23$ (prime modulus), $g = 5$ (primitive root/generator), a (A's private key) = 6, b (B's private key) = 15

d. Compute public keys for A and B

User A's public key: $A = g^a \bmod p = 5^6 \bmod 23$

Compute powers of 5 mod 23 step by step:

- $5^1 = 5$
- $5^2 = 25 \bmod 23 = 2$
- $5^3 = 5^2 \times 5 = 2 \times 5 = 10$
- $5^4 = 5^3 \times 5 = 10 \times 5 = 50 \bmod 23 = 4$
- $5^5 = 5^4 \times 5 = 4 \times 5 = 20$
- $5^6 = 5^5 \times 5 = 20 \times 5 = 100 \bmod 23 = 100 - 4 \times 23 = 100 - 92 = 8$

A = 8

User B's public key: $B = g^b \bmod p = 5^{15} \bmod 23$

Continue the power sequence:

- $5^7 = 5^6 \times 5 = 8 \times 5 = 40 \bmod 23 = 17$
- $5^8 = 5^7 \times 5 = 17 \times 5 = 85 \bmod 23 = 85 - 69 = 16$
- $5^9 = 16 \times 5 = 80 \bmod 23 = 80 - 69 = 11$
- $5^{10} = 11 \times 5 = 55 \bmod 23 = 55 - 46 = 9$
- $5^{11} = 9 \times 5 = 45 \bmod 23 = 45 - 23 = 22$
- $5^{12} = 22 \times 5 = 110 \bmod 23 = 110 - 92 = 18$
- $5^{13} = 18 \times 5 = 90 \bmod 23 = 90 - 69 = 21$
- $5^{14} = 21 \times 5 = 105 \bmod 23 = 105 - 92 = 13$
- $5^{15} = 13 \times 5 = 65 \bmod 23 = 65 - 46 = \mathbf{19}$

B = 19

Public values exchanged over the (insecure) channel: **A sends 8, B sends 19**

e. Compute the shared secret key

Both parties compute the same shared secret using the other's public value raised to their own private key:

A computes: $K = B^a \bmod p = 19^6 \bmod 23$

- $19^2 = 361 \bmod 23 = 361 - 345 = 16$
- $19^3 = 19^2 \times 19 = 16 \times 19 = 304 \bmod 23 = 304 - 299 = 5$
- $19^6 = (19^3)^2 = 5^2 = 25 \bmod 23 = \mathbf{2}$

B computes: $K = A^b \bmod p = 8^{15} \bmod 23$

- $8^2 = 64 \bmod 23 = 64 - 46 = 18$
- $8^3 = 8^2 \times 8 = 18 \times 8 = 144 \bmod 23 = 144 - 138 = 6$
- $8^6 = (8^3)^2 = 6^2 = 36 \bmod 23 = 13$
- $8^7 = 8^6 \times 8 = 13 \times 8 = 104 \bmod 23 = 104 - 92 = 12$
- $8^{14} = (8^7)^2 = 12^2 = 144 \bmod 23 = 144 - 138 = 6$
- $8^{15} = 8^{14} \times 8 = 6 \times 8 = 48 \bmod 23 = 48 - 46 = \mathbf{2}$

Shared secret key K = 2 — both A and B independently arrive at the same value, confirming the protocol works correctly, without ever transmitting a or b (their private keys) over the channel.

f. Man-in-the-Middle (MITM) attack demonstration

Diffie-Hellman, in its **basic/unauthenticated form**, has no mechanism to verify *who* sent a given public value — this is its critical weakness.

Attack sequence:

1. A wants to establish a shared key with B and sends her public value **A = 8** toward B.
2. The attacker, Eve, sitting on the communication channel, **intercepts** this message and does not forward it.
3. Eve generates her own private key, say $e_1 = 4$, computes her own public value $E_a = 5^4 \bmod 23 = 4$, and sends **E_a = 4** to B, pretending it came from A.
4. B, unaware of the interception, replies with his own public value **B = 19**, intended for A.
5. Eve intercepts this too, generates a second private key e_2 (or reuses one), computes another public value E_b , and sends this to A pretending it came from B.
6. Now:
 - Eve and A compute a shared secret K_1 (Eve using e_2 and A's real public value 8; A using her private key 6 and Eve's fake value)
 - Eve and B compute a **separate** shared secret K_2 (Eve using e_1 and B's real public value 19; B using his private key 15 and Eve's fake value)
7. A believes she shares K_1 with B; B believes he shares K_2 with A — but actually **Eve holds both keys**. When A sends an encrypted message using K_1 , Eve decrypts it, reads/modifies it, re-encrypts it using K_2 , and forwards it to B (and vice versa).
8. Neither A nor B detects anything unusual — the exchange completes and communication continues normally, but **Eve has full read/write access** to everything passing between them, entirely undetected.

g. Secure method to prevent the attack

- **Authentication of exchanged values:** Use **digital signatures** — each party signs their public DH value with their private signing key (verified via a certificate/PKI), so the receiver can confirm the value genuinely came from the claimed sender and wasn't substituted.
- **Station-to-Station (STS) Protocol:** An enhanced version of DH that

incorporates digital signatures and a MAC into the exchange specifically to prevent MITM attacks.

- **TLS-style certificate verification:** Bind public keys to identities using certificates issued by a trusted Certificate Authority (CA), so A and B can verify each other's identity before trusting any exchanged value (this is exactly how HTTPS/TLS uses DH or ECDH securely — via "Ephemeral DH" combined with certificate-based authentication).
- **Out-of-band verification:** In smaller-scale systems, comparing key fingerprints through a separate trusted channel (e.g., verbally, or via a pre-shared secret) can also confirm authenticity.

5. ECC vs RSA Comparison (IoT Device Scenario)

Given: RSA key size = 1024 bits, ECC key size = 160 bits (claimed equivalent security), Device = resource-constrained IoT device

d. Analyze computational cost for both systems

- **RSA (1024-bit):**
 - Relies on the difficulty of factoring large integers (Integer Factorization Problem).
 - Core operation is **modular exponentiation** on very large numbers (1024-bit integers), which requires many large-number multiplications — computationally expensive, especially on constrained hardware without dedicated big-number arithmetic units.
 - Key generation is also expensive: it requires finding two large probable primes and verifying primality (e.g., via Miller-Rabin tests), which is a heavier one-time cost.
- **ECC (160-bit):**
 - Relies on the difficulty of the **Elliptic Curve Discrete Logarithm Problem (ECDLP)**, which is significantly harder to solve per bit than integer factorization — this is *why* ECC can achieve equivalent security with a much smaller key size.
 - Core operation is **point multiplication** on the elliptic curve (scalar multiplication of a point), computed via repeated point addition/doubling — this operates on much smaller numbers (160-bit vs 1024-bit), so each operation requires far less computation.

- Key generation is comparatively lightweight — just selecting a random scalar as the private key and computing the corresponding point.

Approximate equivalence table (standard cryptographic reference):

Security level (bits)	RSA key size	ECC key size
80	1024	160
112	2048	224
128	3072	256

This confirms the problem's premise: RSA-1024 $\approx$ ECC-160 in security strength.

e. Estimate power and memory consumption

- **Memory:** ECC keys, signatures, and ciphertexts are dramatically smaller — roughly **1/6th to 1/10th the size** of RSA equivalents at the same security level. For an IoT device with kilobytes (not megabytes) of RAM/flash, this is a major advantage — less memory needed to store keys and less bandwidth to transmit them.
- **Power/processing:** Because ECC point multiplication operates on much smaller numbers than RSA's modular exponentiation on 1024-bit integers, ECC requires significantly fewer CPU cycles per operation. Published benchmarks commonly show ECC operations (at equivalent security) completing several times faster than RSA, and consuming a fraction of the energy — critical for battery-powered/low-power IoT devices where every CPU cycle drains the battery.
- **Conclusion:** **ECC consumes less power and less memory** than RSA for equivalent security.

f. Justification — which cryptosystem is better for this scenario

ECC is the better choice for the IoT device.

Reasoning:

1. **Resource constraints:** IoT devices typically have limited CPU power, small memory footprints, and run on battery power. ECC's smaller key sizes directly translate to less storage needed and faster computation – both critical for such devices.
2. **Equivalent security, smaller footprint:** ECC-160 provides security comparable to RSA-1024, meaning the device doesn't sacrifice security to gain efficiency – it gets the same protection with a much lighter computational load.
3. **Energy efficiency:** Since IoT devices are often battery-operated or energy-harvesting, and cryptographic operations are a significant power draw, ECC's lower computational overhead directly extends battery life / device operability.
4. **Bandwidth savings:** Smaller keys and signatures also mean less data to transmit over the network, which matters for IoT devices often operating over low-bandwidth, low-power wireless protocols (e.g., Zigbee, LoRa, BLE).
5. **RSA's drawback here:** While RSA is well-established and secure at appropriate key sizes, its computational and memory demands (large keys, expensive modular exponentiation) make it a poor fit for constrained hardware – it would either be too slow for real-time operations or drain the battery quickly.

Final verdict: ECC is recommended for the given IoT scenario due to its superior balance of strong security, low computational cost, reduced memory usage, and lower power consumption compared to RSA at an equivalent security level.